Filsystem, del 2

Jimmy Åhlander

2025-12-11

Mittuniversitetet
MID SWEDEN UNIVERSITY

Agenda

- Från fil till data – olika abstraktionslager.
- Caching av filsystemets strukturer.
- Lagring av katalogstrukturen.
- Allokering av block i sekundärminnet:
  - Sammanhängande allokering.
  - Länkad allokering.
  - Indexerad allokering.

Föreläsningen i ett nötskal

- Den här föreläsningen handlar om hur de logiska filerna kopplas fysiskt till block på sekundärminnet.

- Föreläsningen motsvarar kapitel 14 *File-System Implementation* i Operating System Concepts.

Filsystemsstrukturer

Vi har lärt oss att:

- Data lagras i fysiska block om 512 till 4096 bytes på sekundärminnet.
- Filsystemet tillhandahåller ett enkelt och smidigt sätt att komma åt data.
- Det första problemet vi adresserade var hur filsystemet ska se ut för användaren.

Nu:

- Det andra problemet är att skapa algoritmer och datastrukturer för att koppla det logiska filsystemet (som användaren ser) mot det fysiska i sekundärminnet.

Filkontrollblock (*FCB*)

file permissions
file dates (create, access, write)
file owner, group, ACL
file size
file data blocks or pointers to file data blocks

Bild: A. Silberschatz, et al. Operating System Concepts, 10th ed. 2018. Fig. 14.02

Från fil till data – olika abstraktionslager

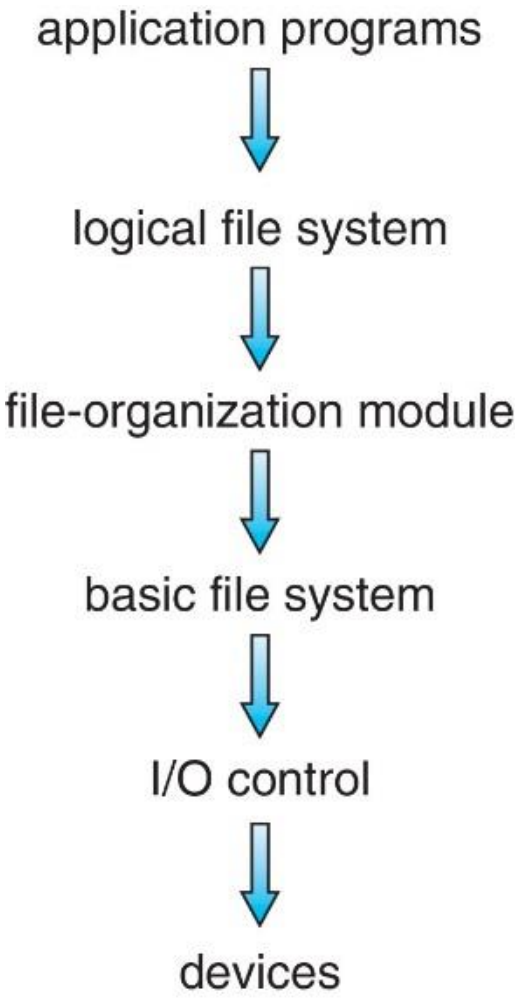


Bild: A. Silberschatz, et al. Operating System Concepts, 10th ed. 2018. Fig. 14.01

Logical file system

- Det här lagret hanterar metadata, det vill säga, allt förutom filens data (filinnehållet).
- Det logiska filsystemet hanterar katalogstrukturen.
- Den översätter symboliska filnamn till ID:n som filorganisationsmodulen behöver.

- Den sparar denna information i filkontrollblock (*FCBs*). I UNIX-lika system kallas dessa *inodes*. I Windows kallas dessa *MFT records*.

- Det här lagret ansvarar också för skydd och säkerhet. *Permissions* hanteras här.

File-organization module

- Det här lagret håller reda på samtliga filer och kopplar dessa mot deras block på den fysiska enheten.
- Filorganisationsmodulen översätter logiska blockadresser till fysiska blockadresser. Den utgår ifrån filens ID.
- Det här lagret spårar också lediga fysiska block på enheten.

Basic file system

- Det grundläggande filsystemet skickar generella kommandon till lämpliga drivrutiner: Läs från block X. Skriv till block Y.
- Det här lagret hanterar bufferts och cacheminnen som bland annat håller olika datablock.

- Block allokeras till bufferten innan överföringen startar.
- Cacheminne används för att öka systemprestandan. De måste hållas uppdaterade för att fungera.

I/O Control

- Det här lagret består av drivrutiner och interrupt handlers.
- Det här lagret överför information mellan primärminnet och sekundärminnet.

- Från övre lagren: `fetch block 123`.
- Till nedre lagren: hårdvaruspecifika instruktioner.

- Drivrutinen skriver specifika bitmönster till speciella platser i I/O-controllerns minne.

Volymer och boot

- En volym innehåller ett *boot control block* och ett *volume control block*.

Block \ FS	**UFS (ext4, m.fl)**	**NTFS**
Boot control block	Boot block	(Partition) boot sector
Volume control block	Superblock	Master file table (MFT)

Filsystemet cachas

- Data om filsystemet cachas i primärminnet.

- Data laddas in när filsystemet monteras.
- Data uppdateras under användning.
- Data förkastas när filsystemet avmonteras.

Filsystemets strukturer i kärnan

- OS håller en tabell över monterade volymer i minnet, se `mount(8)`.

- Vidare cachas katalogstrukturen med nyligen åtkomna kataloger. Testa:
  `time find /some/subdir -print > /dev/null`
  två gånger i rad.

Ett test …

```
\$ time find ~ -print > /dev/null
real     0m47.535s
user     0m0.479s
sys      0m1.779s
\$
```

```
\$ time find ~ -print > /dev/null
real     0m0.180s
user     0m0.080s
sys      0m0.100s
\$
```

Lagring av katalogstrukturen

Vi måste lagra vår katalogstruktur i någon datastruktur.

Olika datastrukturer som kan användas:

- Linjära listor.
- Hashtabeller.

Lagring av katalogstrukturen
Linjär lista

- Den linjära listan är simpel.
- Filinformation lagras i en lista från början till slut.

- För att skapa en fil itererar vi genom listan för att se att namnet inte är upptaget och därefter skapas ett nytt inlägg.

- För att ta bort en fil frigörs inlägget, exempelvis genom att sätta den till `NULL` eller genom att ersätta den med det sista inlägget och reducera längden på listan ett steg.

Lagring av katalogstrukturen
Hashtabell

- Linjära listor är kostsamma att använda då de måste genomsökas i linjär tid.
  - Det går att förenkla genom att sortera listan. Problemet blir då istället att hålla listan sorterad.
- Istället kan en hashtabell användas. En hashfunktion beräknar ett hashvärde utifrån det symboliska filnamnet.
- Problemet med hashtabeller är kollisioner. En bra hashfunktion genererar få kollisioner.

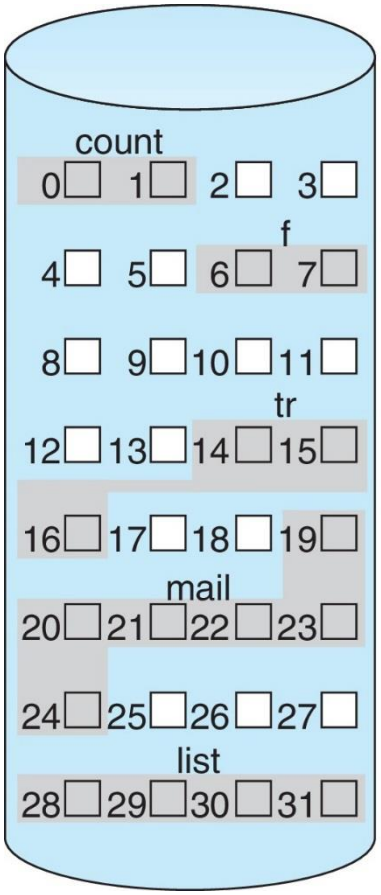
Blockallokering

- Filinnehållet lagras i block på sekundärminnet.
  - Hur håller vi koll på blocken?

Olika algoritmer för blockallokering:

- Sammanhängande allokering.
- Länkad allokering.
- Indexerad allokering.
  - Hierarkisk.

Sammanhängande allokering



directory

file	start	length
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2

Bild: A. Silberschatz, et al. Operating System Concepts, 10th ed. 2018. Fig. 14.04

Länkad allokering

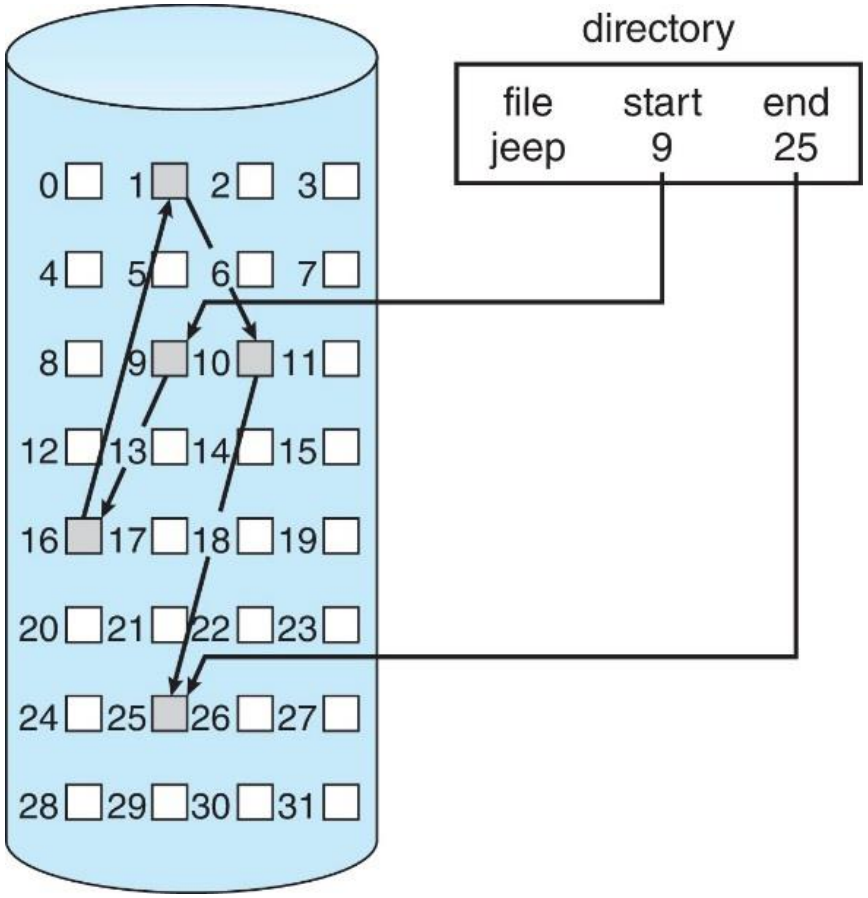


Bild: A. Silberschatz, et al. Operating System Concepts, 10th ed. 2018. Fig. 14.05

Indexerad allokering

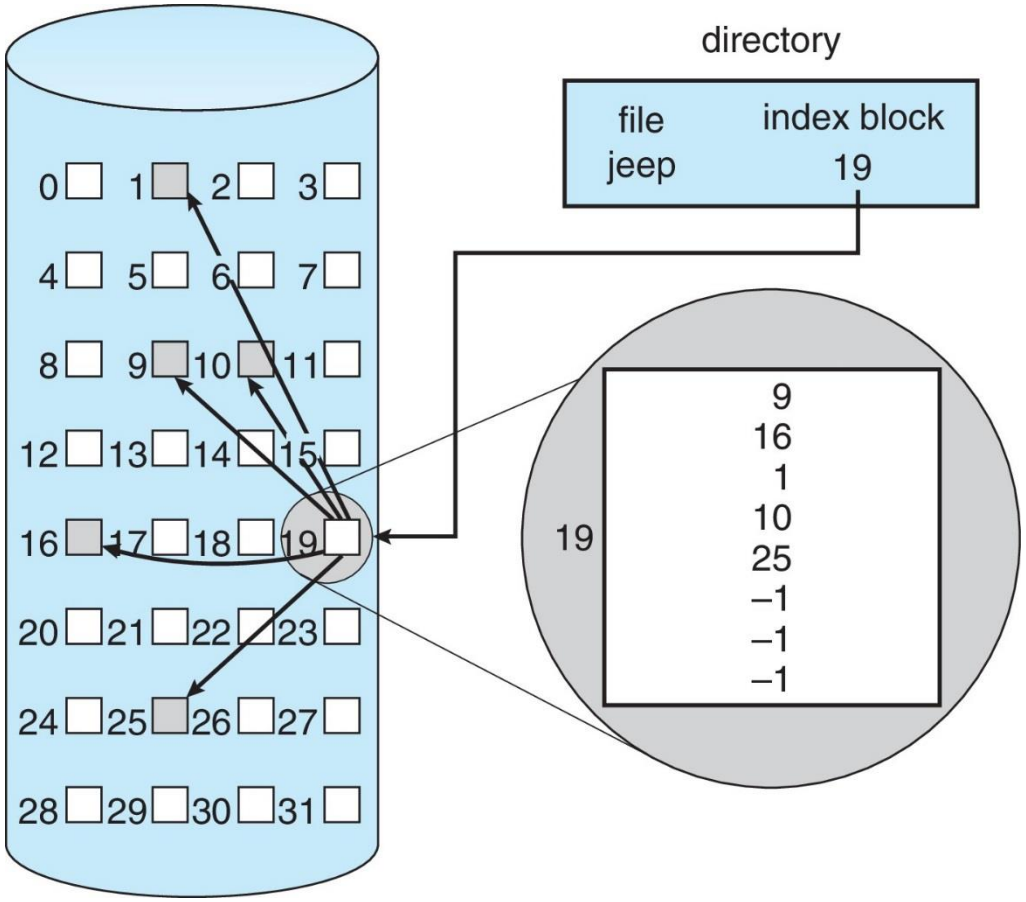


Bild: A. Silberschatz, et al. Operating System Concepts, 10th ed. 2018. Fig. 14.07

Hierarkisk indexerad allokering (för inodes)

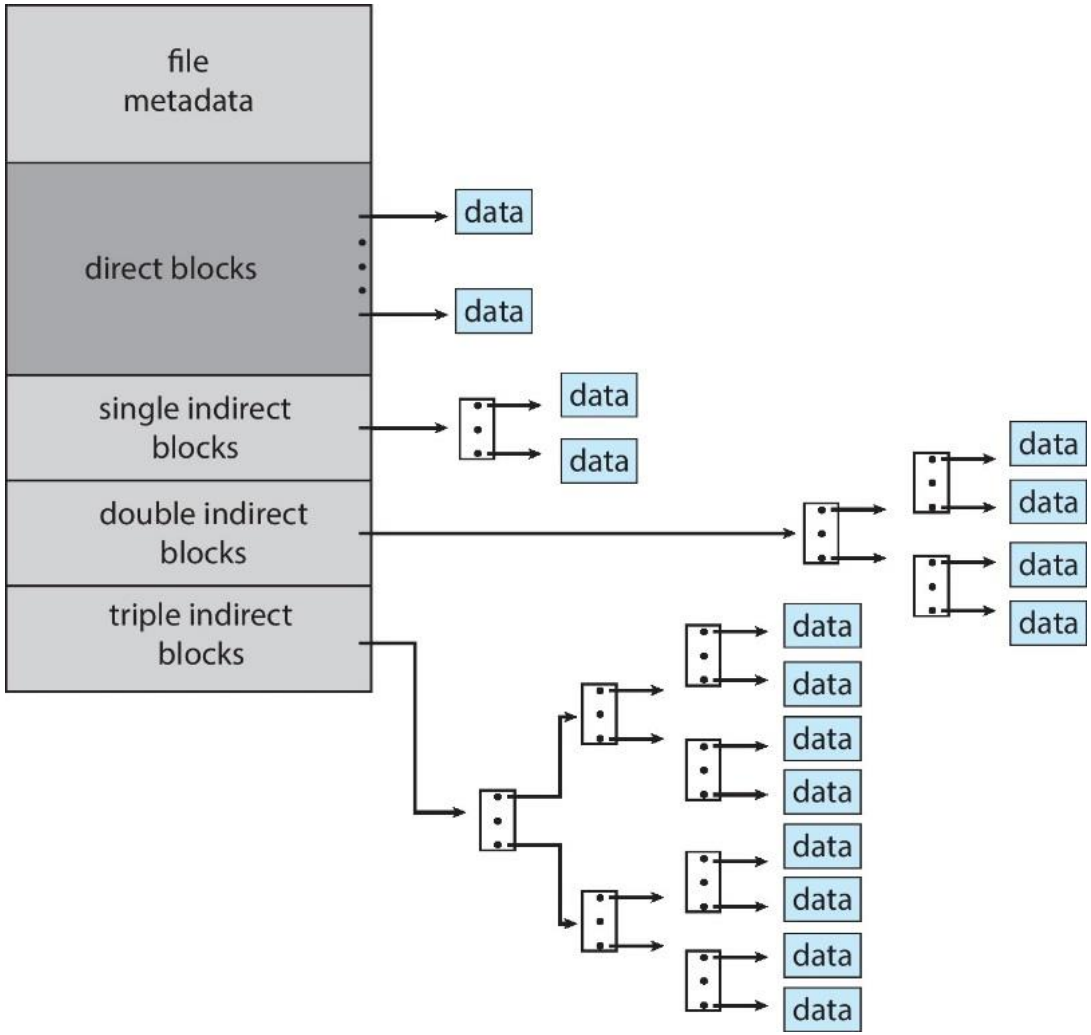


Bild: A. Silberschatz, et al. Operating System Concepts, 10th ed. 2018. Fig. 14.08

Filsystem, del 2
Sammanfattning

- Från en logisk fil till data på sekundärminnet finns flera abstraktionslager med olika ansvar.
- För blockallokering kan vi tillämpa olika algoritmer för olika resultat.